

Q1 Perform and Write Truffle installation Steps

Ans

Installing Truffle on Windows is a straightforward process, but it requires a few prerequisites to ensure the development environment runs smoothly. Truffle is a Node.js-based framework, so having the right version of Node is the most critical step.

Step 1: Install Node.js

Truffle requires **Node.js** to function.

1. Visit the official [Node.js website](https://nodejs.org/).
2. Download the **LTS (Long Term Support)** version for Windows.
3. Run the installer and follow the prompts. Ensure that the option to "**Add to PATH**" is checked.
4. **Verify the installation:** Open your Command Prompt (CMD) or PowerShell and type:

Bash

```
node -v
```

```
npm -v
```

You should see version numbers for both.

Step 2: Install Windows Build Tools (Optional but Recommended)

Some Truffle dependencies require C++ compilation. If you encounter errors during installation, you may need the native build tools.

Open **PowerShell as an Administrator** and run:

PowerShell

```
npm install --global windows-build-tools
```

Step 3: Install Truffle Globally

Once Node.js and NPM are ready, you can install Truffle using the following command in your terminal:

Bash

```
npm install -g truffle
```

Note: The -g flag ensures Truffle is installed globally so you can use the truffle command in any folder.

Verify Truffle Installation

After the installation completes, check the version to confirm success:

Bash
truffle version

Step 4: Resolve "Execution Policy" Errors (PowerShell Only)

If you are using PowerShell and get an error saying *"truffle.ps1 cannot be loaded because running scripts is disabled,"* you need to update your execution policy:

1. Open **PowerShell as Administrator**.
2. Run the following command:

PowerShell

Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser

3. Type Y and hit Enter.

Step 5: Initialize a New Project

To start using Truffle, create a new directory for your blockchain project and initialize it:

1. **Create folder:** `mkdir my-dapp && cd my-dapp`
2. **Initialize:** `truffle init`

Initial Directory Structure:

- **contracts/**: Where your Solidity smart contracts live.
- **migrations/**: Scriptable deployment files.
- **test/**: For testing your application and contracts.
- **truffle-config.js**: Your configuration file for networks and compilers.

Q2 Perform and write Simple program using solidity language and deployment on truffle local blockchain

To perform this task, you will combine your earlier **Truffle** installation with a new Solidity contract. Since you already have Truffle on your Windows machine, you will now move from the browser-based Remix environment to a professional local development workflow.

Step 1: Initialize Your Truffle Project

Open your Command Prompt or PowerShell and create a dedicated workspace for this program.

PowerShell

```
mkdir truffle-demo && cd truffle-demo  
truffle init
```

This command creates the standard directory structure: `contracts/`, `migrations/`, `test/`, and `truffle-config.js`.



Step 2: Write the Solidity Smart Contract

Create a file named **SimpleStorage.sol** inside the `contracts/` folder. We will use the same logic you successfully compiled earlier in Remix.

Solidity

```
// SPDX-License-Identifier: MIT  
pragma solidity ^0.8.0;  
  
contract SimpleStorage {  
    uint256 public storedData;  
  
    // Function to update the value  
    function set(uint256 x) public {  
        storedData = x;  
    }  
  
    // Function to read the value  
    function get() public view returns (uint256) {  
        return storedData;  
    }  
}
```



Step 3: Create the Deployment Script

Truffle needs a "migration" script to know how to deploy your specific contract. Create a file named **1_deploy_contract.js** inside the `migrations/` folder.

JavaScript

```
const SimpleStorage = artifacts.require("SimpleStorage");  
  
module.exports = function (deployer) {  
    deployer.deploy(SimpleStorage);  
};
```



Step 4: Configure the Local Blockchain

You need a local blockchain to deploy to. You have two main options:

1. **Truffle Develop:** Built directly into Truffle. Run `truffle develop` in your terminal to start a local node on port 9545.
2. **Ganache:** A separate GUI tool (which you previously looked at) that runs on port 7545.

```

Last login: Wed May 13 19:32:32 on ttys000
prathamved@MacBook-Air ~ % mkdir truffle-demo && cd truffle-demo
truffle init

Starting init...
=====
> Copying project files to /Users/prathamved/truffle-demo

Init successful, sweet!

Try our scaffold commands to get started:
$ truffle create contract YourContractName # scaffold a contract
$ truffle create test YourTestName        # scaffold a test

http://trufflesuite.com/docs

prathamved@MacBook-Air truffle-demo % truffle develop
Truffle Develop started at http://127.0.0.1:9545/

Accounts:
(0) 0x921ef340e84bf823e0fe524071f04a19db451a8e
(1) 0x611c4aed24a605390f5b65e8edf398b1e103c81e
(2) 0x8555d4b524385e0f6679e74cd37dd02f6fcd44c
(3) 0x7aa9c8725314fee5b5dec16208a0c7f035e9ef
(4) 0x20b4db7d45183c08c49abfa7bc7d0e7c822fbaf
(5) 0xd8ccf280baf63b2f79292bb4bc2f712bbdf10014
(6) 0xa788bb085a81a50f888e103d7d8d1a1e87e68ce2
(7) 0x7cfc14f5628fb5416f50a7044488ec259e0a52d
(8) 0x47756342f1c553d5c39606d1d55467e04ab2fbd
(9) 0xa276a30d5c9532b363f3da1902b5928f0b05bd3b

Private Keys:
(0) 573990571ae67eb2866547c34b0fe40a4e95819b502d3af1a47a25cb0b78729
(1) 0c8a90ed22b8b4d2adeec171dc89ce91e9b50bf9f6f090589a46085d8603f211
(2) ee29189275dbe6d78218e7425aed247a8c8ecdd23c9097e7c2d30c61dfa19bf1
(3) f2f3b6560565e4478609d68160f7d0efb0437013fabfd638af78a5d0c30b916a

```

If using **Ganache**, ensure your truffle-config.js has these lines uncommented:

JavaScript

```

module.exports = {
  networks: {
    development: {
      host: "127.0.0.1",
      port: 7545,
      network_id: "*",
    },
  },

  compilers: {
    solc: {
      version: "0.8.21",
      settings: {
        optimizer: {
          enabled: false,
          runs: 200
        },
        evmVersion: "london"
      }
    }
  },
};

```



Step 5: Compile and Deploy

Run the following commands in your terminal:

1. Compile the contract:

PowerShell

truffle compile

2. Migrate (Deploy) to the local network:

PowerShell

truffle migrate

Step 6: Verify the Deployment

You can interact with your deployed contract directly through the **Truffle Console** to prove it works:

PowerShell

```
truffle console --network development
```

Once inside the console, run these commands:

JavaScript

```
const instance = await SimpleStorage.deployed()
```

```
await instance.set(64)
```

```
const res = await instance.get()
```

```
res.toString()
```

[illegible]

Q3 To install MetaMask on your browser (Chrome, Firefox, Edge, or Brave) or mobile device, follow these steps:

Browser Extension (Chrome, Firefox, Brave, Edge)

1. **Visit the Official Site:** Go to metamask.io/download. Always verify the URL to avoid fraudulent sites.
2. **Add to Browser:** Click "**Download**" and select your specific browser.
 - **Chrome:** Click "Add to Chrome" followed by "Add extension" in the pop-up.
 - **Firefox:** Click "Add to Firefox" and then "Add" in the pop-up.
 - **Edge:** Select the Edge icon and follow the prompts to install.
3. **Launch:** Once installed, the orange fox icon will appear in your browser's extension toolbar.

Mobile App (iOS and Android)

1. **Official Stores:** Open the **App Store (iOS)** or **Google Play Store (Android)** and search for "MetaMask".
2. **Verify Publisher:** Ensure the publisher is **ConsenSys** to avoid malicious clones.
3. **Install:** Tap "Get" or "Install" and wait for the process to finish.

Wallet Setup & Security

After installation, you must set up your wallet. This is the most critical step for your security:

1. **Create a New Wallet:** Click "Create a New Wallet" on the welcome screen.
2. **Set a Local Password:** Create a strong password to encrypt your wallet on that specific device.
3. **Secret Recovery Phrase (SRP):** MetaMask will generate a **12-word recovery phrase**.
 - **Crucial Rule:** Write these 12 words down on physical paper and store them in a secure, offline location.
 - **Warning:** Never share this phrase with anyone. If you lose it, your funds cannot be recovered.
4. **Confirm Phrase:** Complete the short quiz by selecting the 12 words in the correct order to verify you have saved them.

Using MetaMask for Development

Since you are working with **Truffle** and **Remix**, you will often need to add custom networks:

- **Add Network:** Click the network selector at the top (usually "Ethereum Mainnet") and select "Add Network".
- **Manual Entry:** You can manually enter details like **RPC URL**, **Chain ID**, and **Currency Symbol** to connect to local blockchains like Ganache or testnets like Sepolia.

[How to Use MetaMask In 2026 \(FULL GUIDE\)](#)

This video provides a complete walkthrough of installing and using MetaMask, including funding and sending transactions in 2026.

Q4 Write Smart contract using Metamask and Remix

Since you've already got Truffle ready to go, moving over to **Remix IDE** is a great next step. It's a powerful web-based tool that handles the heavy lifting of compilation and deployment without you having to touch the command line.

Here is how you can write, compile, and deploy a smart contract using **Remix** and **MetaMask**.

Step 1: Set Up MetaMask

Before heading to Remix, you need a wallet to sign transactions.

1. **Install Extension:** Add the [MetaMask](#) extension to your browser.
2. **Create/Unlock Wallet:** Set up your password and recovery phrase.
3. **Select a Network:** Switch to a **Testnet** (like Sepolia or Holesky) so you don't spend real money.
 - *Note: You will need some "Test ETH" from a faucet to pay for gas.*
4. Add test ETH using a faucet.

<https://cloud.google.com/application/web3/faucet/ethereum/sepolia>

Step 2: Write the Smart Contract in Remix

1. Go to remix.ethereum.org.
2. In the **File Explorer** (top left icon), click the **File icon** to create a new file. Name it SimpleStorage.sol.
3. Copy and paste this basic Solidity code:

Solidity

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
```

```
contract SimpleStorage {
    uint256 public favoriteNumber;

    // Function to update the number
    function store(uint256 _number) public {
        favoriteNumber = _number;
    }
}
```

```
}  
}
```

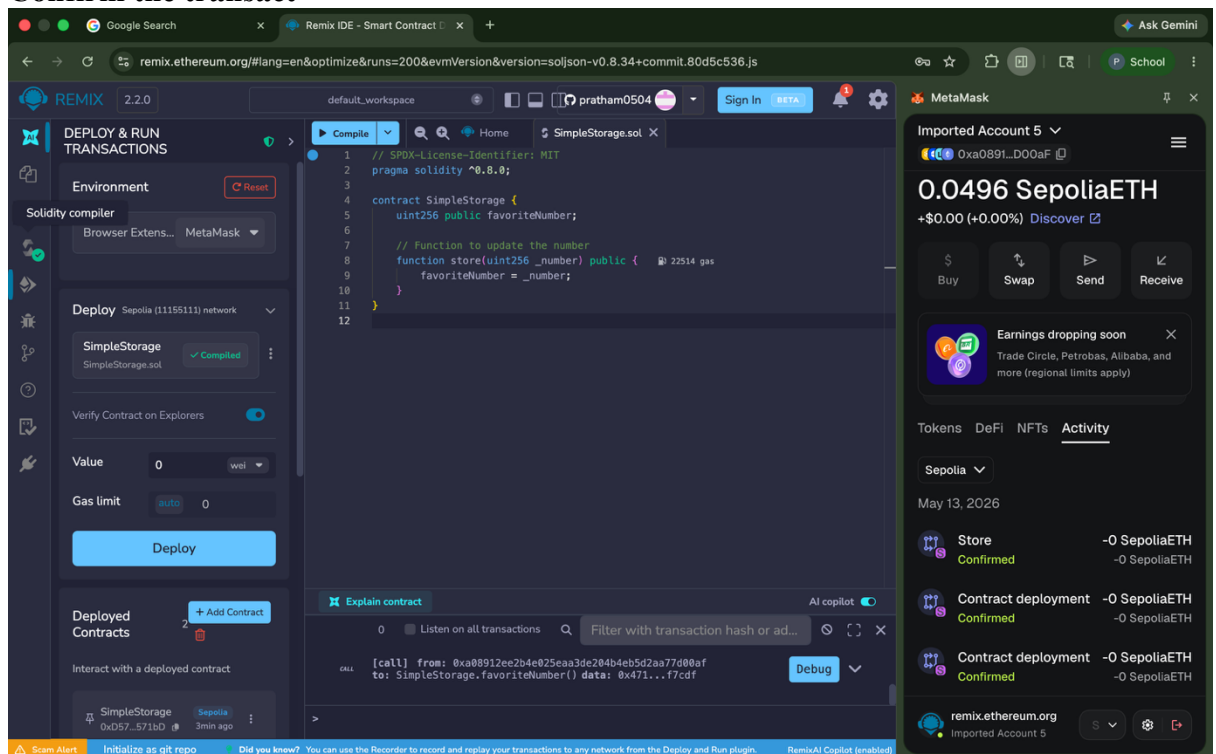
Step 3: Compile the Contract

1. Click the **Solidity Compiler** icon on the left sidebar (it looks like a small "S" or a hexagon).
2. Ensure the **Compiler version** matches the one in your code (e.g., 0.8.0).
3. Click the blue **Compile SimpleStorage.sol** button.
 - o *A green checkmark will appear if there are no errors.*

Step 4: Connect MetaMask and Deploy

This is where the magic happens. We need to tell Remix to talk to your MetaMask wallet.

1. Click the **Deploy & Run Transactions** icon (it looks like an Ethereum logo with a play button).
2. Under **Environment**, click the dropdown and select **browser - MetaMask**.
3. A MetaMask popup will appear; click **Connect** to link your wallet to Remix.
4. Ensure your contract (SimpleStorage) is selected in the **Contract** dropdown.
5. Click the orange **Deploy** button.
6. **Confirm the transact**

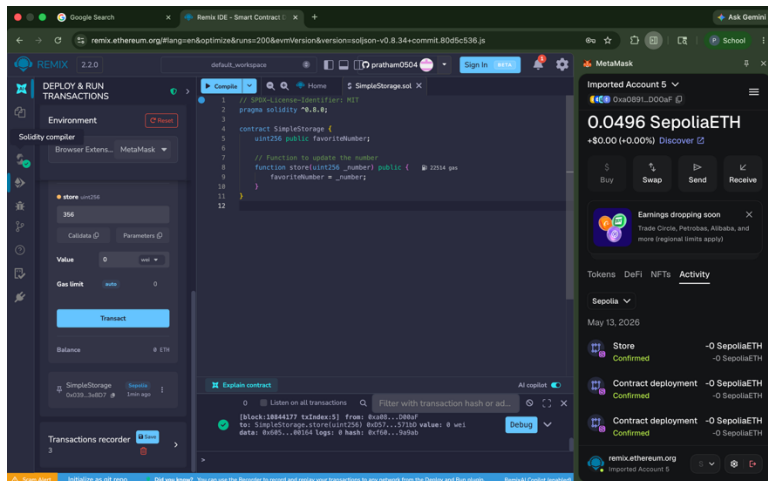


ion in the MetaMask popup that appears.

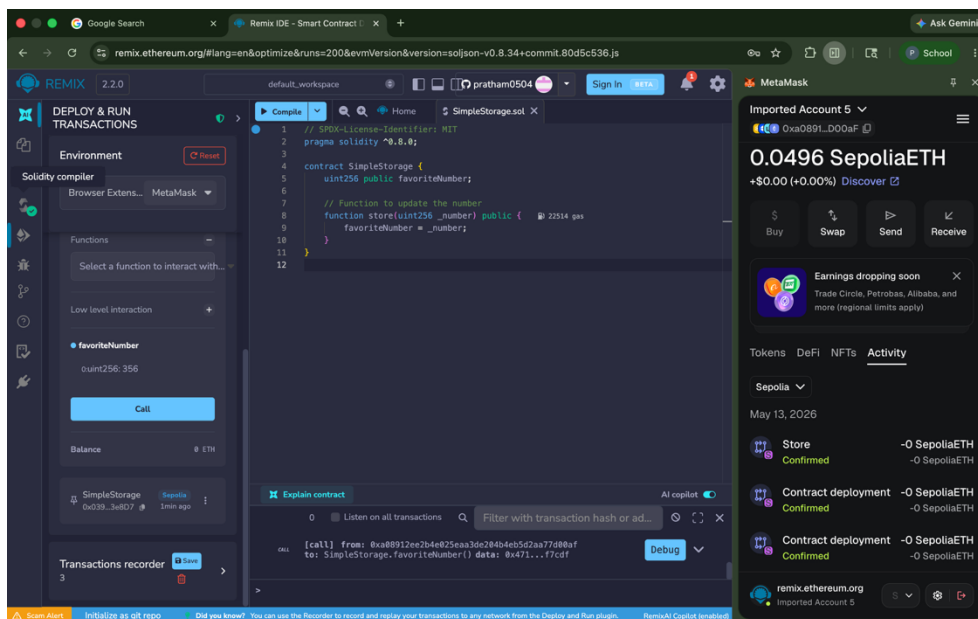
Step 5: Interact with the Contract

Once the transaction is confirmed on the blockchain, your contract will appear under **Deployed Contracts** at the bottom of the sidebar.

- **Expand the contract** by clicking the arrow next to its name.
- **Test the Store function:** Enter a number in the box next to store and click the button. (This costs gas, so MetaMask will pop up again).



- **Test the Fetch function:** Click the favoriteNumber button to see the value you just saved. It should update to your number!



Q5 Perform and write web3 program

Perform and Write Web3 Program

A **Web3 program** allows interaction between a web application and the Ethereum blockchain using JavaScript and the Web3.js library.

Aim

To create a simple Web3 program that connects to Ethereum using MetaMask and displays the connected wallet address.

Requirements

- Node.js installed
- MetaMask browser extension
- Web browser
- Internet connection
- Web3.js library

Step 1: Create Project Folder

Create a new folder for the project.

```
mkdir web3project  
cd web3project
```

Step 2: Initialize Node Project

Run:

```
npm init -y
```

Step 3: Install Web3.js

Install the Web3 library:

```
npm install web3
```

```

npm audit fix

To address all issues (including breaking changes), run:
npm audit fix --force

Run `npm audit` for details.
prathamved@MacBook-Air storage-dapp % mkdir web3project
cd web3project
prathamved@MacBook-Air web3project % cd ..
prathamved@MacBook-Air storage-dapp % cd .
prathamved@MacBook-Air storage-dapp % cd..
zsh: command not found: cd..
prathamved@MacBook-Air storage-dapp % cd ..
prathamved@MacBook-Air .Trash % cd ..
prathamved@MacBook-Air ~ % mkdir web3project
cd web3project
prathamved@MacBook-Air web3project % npm init -y
Wrote to /Users/prathamved/web3project/package.json:

{
  "name": "web3project",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "description": ""
}

prathamved@MacBook-Air web3project % npm install web3

added 75 packages, and audited 76 packages in 4s

```

Step 4: Create HTML File

Create a file named index.html

```

<!DOCTYPE html>
<html>
<head>
  <title>Web3 Demo</title>
</head>
<body>
  <h2>Web3 Ethereum Connection</h2>
  <button onclick="connectWallet()">Connect MetaMask</button>
  <p id="wallet"></p>

  <script src="https://cdn.jsdelivr.net/npm/web3@latest/dist/web3.min.js"></script>

  <script>
    async function connectWallet() {
      if (window.ethereum) {
        try {
          await window.ethereum.request({ method: 'eth_requestAccounts' });

          const web3 = new Web3(window.ethereum);
          const accounts = await web3.eth.getAccounts();

          document.getElementById("wallet").innerHTML =
            "Connected Wallet: " + accounts[0];

        } catch (error) {

```

```
        console.log(error);
    }
} else {
    alert("MetaMask not installed");
}
}
</script>
</body>
</html>
```

Step 5: Run the Program

Open the HTML file in a browser.

Step 6: Execute

1. Open browser
2. Ensure MetaMask is installed
3. Click **Connect MetaMask**
4. Approve connection in MetaMask
5. Wallet address will be displayed

Expected Output

Example:

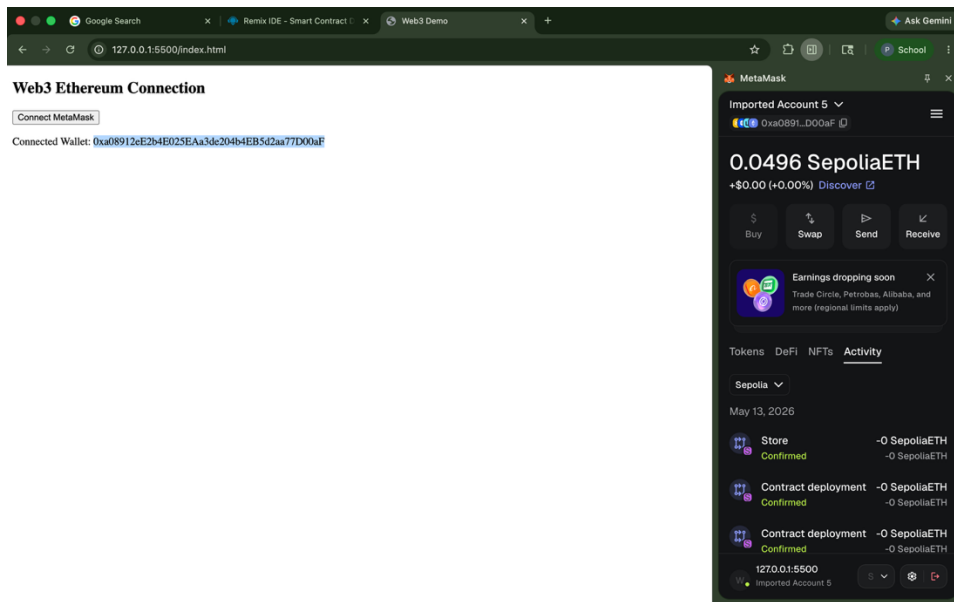
Connected Wallet: 0xA34F6C9D12E78B56D90A12F34B67C89D12345678

Explanation

- window.ethereum checks whether MetaMask is installed
- eth_requestAccounts requests wallet access
- Web3(window.ethereum) creates blockchain connection
- getAccounts() fetches wallet addresses

Result

The Web3 program successfully connects the browser application to Ethereum using MetaMask and displays the user's wallet address.



Q6 Perform and write study of Cryptocurrency development.

Design and Develop Cryptocurrency

◆ PART A: Install and Configure MetaMask

Step 1: Install MetaMask

1. Open Google Chrome
2. Visit:

<https://metamask.io>

3. Click **Download**
4. Click **Add to Chrome**
5. Click **Add Extension**

Step 2: Create Wallet

1. Open MetaMask
 2. Click **Create a Wallet**
 3. Create password
 4. Save the **12-word Secret Recovery Phrase**
 5. Complete setup
-

Step 3: Enable Sepolia Network

1. Open MetaMask
2. Click network dropdown at top
3. Select:

Sepolia Test Network

If not visible:

- Go to Settings → Advanced
- Enable:

Show Test Networks

Step 4: Get Test ETH

1. Copy MetaMask wallet address
 2. Open Sepolia Faucet website
 3. Paste address
 4. Request test ETH
 5. Wait for ETH to appear in wallet
-

◆ PART B: Open Remix IDE

Step 5: Open Remix

Visit:

<https://remix.ethereum.org>

◆ PART C: Create Cryptocurrency Smart Contract

Step 6: Create Solidity File

1. Open contracts folder
2. Click **New File**
3. Create:

MyToken.sol

Step 7: Write Smart Contract Code

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract MyToken {

    string public name = "PurvaCoin";
    string public symbol = "PVC";
    uint8 public decimals = 18;
    uint256 public totalSupply;

    mapping(address => uint256) public balanceOf;

    constructor(uint256 _initialSupply) {
        totalSupply = _initialSupply;
        balanceOf[msg.sender] = _initialSupply;
    }

    function transfer(address _to, uint256 _value) public returns (bool success) {

        require(balanceOf[msg.sender] >= _value, "Insufficient balance");

        balanceOf[msg.sender] -= _value;
        balanceOf[_to] += _value;

        return true;
    }
}
```

◆ PART D: Compile Smart Contract

Step 8: Compile Contract

1. Click **Solidity Compiler**
2. Select compiler version:

0.8.20

3. Click:

Compile MyToken.sol

✓ If no errors appear, compilation is successful.

◆ PART E: Deploy Smart Contract

Step 9: Open Deploy Section

1. Click:

Deploy & Run Transactions

Step 10: Select Environment

Under Environment choose:

Injected Provider - MetaMask

👉 MetaMask popup appears.

Step 11: Connect MetaMask

1. Click: Next
 2. Click: Connect
 3. Ensure network is: Sepolia
-

Step 12: Enter Initial Supply

In constructor parameter field:

`_initialSupply`

Enter:

1000

Step 13: Deploy Contract

1. Click:

Deploy

2. MetaMask popup appears
3. Click:

Confirm

✓ Contract deploys successfully.

◆ PART F: Locate Deployed Contract

Step 14: Open Deployed Contract

1. Scroll down to: Deployed Contracts
 2. You will see: MyToken at 0x.....
 3. Click arrow:  to expand the contract.
-

◆ PART G: Check Total Supply

Step 15: Check totalSupply

1. Click: **totalSupply**
 2. Output: 1000
 3. ✓ This indicates total tokens created.
-

◆ PART H: Check Account Balance

Step 16: Copy Account 1 Address

1. Open MetaMask
2. Select: Account 1
3. Copy wallet address

Step 17: Check balanceOf

1. In Remix find: **balanceOf**
2. Paste Account 1 address in: param0(address)
3. Click: balanceOf
4. Output: 1000

✓ This indicates Account 1 owns all tokens.

◆ PART I: Create Second Account

Step 18: Create Account 2

1. Open MetaMask
 2. Click account icon
 3. Click: Add Account / Create Account
 4. Account 2 is created
-

◆ PART J: Transfer Cryptocurrency

Step 19: Copy Account 2 Address

1. Open MetaMask
 2. Select: Account 2
 3. Copy Account 2 address
-

Step 20: Transfer Tokens

1. In Remix locate:

transfer(address _to, uint256 _value)

2. In _to Paste: Account 2 address
3. In _value Enter: 100
4. Click: transfer
5. MetaMask popup appears
6. Click: Confirm

✓ Transfer successful.

◆ PART K: Verify Updated Balances

Step 21: Check Account 1 Balance

1. Paste Account 1 address in **balanceOf**
2. Click button

Output:

900

Step 22: Check Account 2 Balance

1. Paste Account 2 address
2. Click **balanceOf**

Output: 100

✓ Transfer verified successfully.

OUTPUT

totalSupply = 1000

Account 1 balance = 1000

After transfer:

Account 1 balance = 900

Account 2 balance = 100

Q7 Perform and write study Chain code in Hyperledger Fabric

Q9 Decentralized Voting System

Problem Statement:

Design and implement a smart contract in Remix that allows users to vote for candidates in a secure and transparent election. The system should ensure one vote per address and allow the admin to start and end the voting period.

Smart Contract Implementation (Solidity)

You can copy this code directly into **Remix IDE**. It includes features for candidate registration, voting restrictions, and administrative control.

Solidity

// SPDX-License-Identifier: MIT

pragma solidity ^0.8.0;

```
contract VotingSystem {
    struct Candidate {
        uint id;
        string name;
        uint voteCount;
    }
}
```

```

address public admin;
bool public votingActive;

mapping(uint => Candidate) public candidates;
mapping(address => bool) public hasVoted;
uint public candidatesCount;

event Voted(uint indexed candidateId, address voter);
event VotingStarted();
event VotingEnded();

modifier onlyAdmin() {
    require(msg.sender == admin, "Only admin can perform this action");
    _;
}

constructor() {
    admin = msg.sender;
    votingActive = false;
}

// Admin adds candidates before voting starts
function addCandidate(string memory _name) public onlyAdmin {
    require(!votingActive, "Cannot add candidates during active voting");
    candidatesCount++;
    candidates[candidatesCount] = Candidate(candidatesCount, _name, 0);
}

function startVoting() public onlyAdmin {
    votingActive = true;
    emit VotingStarted();
}

function endVoting() public onlyAdmin {
    votingActive = false;
    emit VotingEnded();
}

function vote(uint _candidateId) public {
    require(votingActive, "Voting is not currently active");
    require(!hasVoted[msg.sender], "You have already voted");
    require(_candidateId > 0 && _candidateId <= candidatesCount, "Invalid candidate ID");

    hasVoted[msg.sender] = true;
    candidates[_candidateId].voteCount++;

    emit Voted(_candidateId, msg.sender);
}

function getVotes(uint _candidateId) public view returns (uint) {

```

```

        return candidates[_candidateId].voteCount;
    }
}

```

✂ Step-by-Step Implementation in Remix

1. Setup and Compilation

- Open [Remix IDE](#).
- Create a new file named Voting.sol and paste the code above.
- Go to the **Solidity Compiler** tab and click **Compile Voting.sol**.

2. Deployment

- Go to the **Deploy & Run Transactions** tab.
- Select **Remix VM** for instant local testing or **Injected Provider - MetaMask** for testnet deployment.
- Click **Deploy**.

3. Administrative Setup (The "Admin" Flow)

- **Add Candidates:** Under "Deployed Contracts," find addCandidate. Enter names like "Alice" or "Bob" and click the orange button for each.
- **Start Election:** Click the startVoting button. Users can no longer be added, and voting is now enabled.

4. The Voting Process (The "User" Flow)

- **Cast Vote:** Users enter the ID of their candidate (e.g., 1) into the vote field and click the button.
- **Security Check:** If the same address tries to click vote again, the transaction will fail with the error: *"You have already voted."*

Q9 Crowdfunding Platform

Problem Statement:

Create a smart contract that allows users to create fundraising campaigns and accept donations. If the funding goal is not met by the deadline, donors should be able to withdraw their funds.

Q10 Decentralized To-Do List

Problem Statement:

Develop a smart contract that allows users to add, update, and mark tasks as completed.

Each user's tasks should be stored independently and be accessible only by the user.

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

```

```

contract Crowdfunding {

    struct Campaign {
        address payable creator;
        string title;
        uint goal;
        uint deadline;
        uint amountCollected;
        bool fundsWithdrawn;
    }

    uint public campaignCount;

    mapping(uint => Campaign) public campaigns;
    mapping(uint => mapping(address => uint)) public donations;

    // Create a new campaign
    function createCampaign(
        string memory _title,
        uint _goal,
        uint _durationInMinutes
    ) public {

        campaignCount++;

        campaigns[campaignCount] = Campaign({
            creator: payable(msg.sender),
            title: _title,
            goal: _goal,
            deadline: block.timestamp + (_durationInMinutes * 1 minutes),
            amountCollected: 0,
            fundsWithdrawn: false
        });
    }

    // Donate to campaign
    function donate(uint _campaignId) public payable {

        Campaign storage campaign = campaigns[_campaignId];

        require(block.timestamp < campaign.deadline, "Campaign ended");
        require(msg.value > 0, "Send some ether");

        donations[_campaignId][msg.sender] += msg.value;
        campaign.amountCollected += msg.value;
    }

    // Withdraw funds by creator if goal reached
    function withdrawFunds(uint _campaignId) public {

```

```

Campaign storage campaign = campaigns[_campaignId];

require(msg.sender == campaign.creator, "Only creator can withdraw");
require(block.timestamp >= campaign.deadline, "Campaign still active");
require(campaign.amountCollected >= campaign.goal, "Goal not reached");
require(!campaign.fundsWithdrawn, "Funds already withdrawn");

campaign.fundsWithdrawn = true;

campaign.creator.transfer(campaign.amountCollected);
}

// Refund donors if goal not reached
function refund(uint _campaignId) public {

    Campaign storage campaign = campaigns[_campaignId];

    require(block.timestamp >= campaign.deadline, "Campaign still active");
    require(campaign.amountCollected < campaign.goal, "Goal was reached");

    uint donatedAmount = donations[_campaignId][msg.sender];

    require(donatedAmount > 0, "No donation found");

    donations[_campaignId][msg.sender] = 0;

    payable(msg.sender).transfer(donatedAmount);
}

// View campaign details
function getCampaign(uint _campaignId)
    public
    view
    returns (
        string memory,
        uint,
        uint,
        uint
    )
{
    Campaign memory campaign = campaigns[_campaignId];

    return (
        campaign.title,
        campaign.goal,
        campaign.deadline,
        campaign.amountCollected
    );
}

```

}

<!-- Steps to Perform in Remix IDE

1. Open Remix IDE

2. Create Solidity File

Crowdfunding.sol

Paste the above code.

3. Compile the Contract

4. Deploy the Contract

Practical Execution

Step 1: Create Campaign

Input:

_title = Help Students

_goal = 5 ether

_durationInMinutes = 5

Click:

createCampaign

Transaction will be successful.

Step 2: Donate Ether

Select another account in MetaMask

Enter:

campaignId = 1

In VALUE field enter:

1 ether

Click: donate

Repeat donations from multiple accounts.

Step 3A: Goal Achieved

If total donations $\geq$ goal:

Wait until deadline ends

Creator account clicks:

withdrawFunds(1)

Funds transfer to creator.

Step 3B: Goal Not Achieved

If donations < goal:
After deadline

Donor clicks:
refund(1)
Donor receives refund.

VIVA QUESTIONS:

1. What is crowdfunding in blockchain?
A decentralized method of raising funds using smart contracts.
2. Why use smart contracts for crowdfunding?
They provide transparency, automation, and security without intermediaries.
3. What is msg.sender?
Address of the user calling the function.
4. What is payable?
Allows a function or address to send/receive Ether.
5. What is block.timestamp?
Current block time used for deadlines.
6. Why is refund functionality important?
It protects donors if funding goals are not achieved. -->

Q11 Simple Bank (Wallet System)

Problem Statement:

Build a smart contract that functions as a simple bank. Users should be able to deposit, withdraw, and check their balance. Only the account holder should have access to their funds.

```
// SPDX-License-Identifier: MIT  
pragma solidity ^0.8.0;
```

```
contract SimpleBank {  
  
    mapping(address => uint) private balances;  
  
    // Deposit Ether  
    function deposit() public payable {  
        require(msg.value > 0, "Deposit amount must be greater than 0");  
  
        balances[msg.sender] += msg.value;  
    }  
}
```

```

    }

    // Withdraw in Ether
    function withdraw(uint amountInEther) public {

        uint amount = amountInEther * 1 ether;

        require(balances[msg.sender] >= amount, "Insufficient balance");

        balances[msg.sender] -= amount;

        payable(msg.sender).transfer(amount);
    }

    // Check balance in Ether
    function checkBalance() public view returns(uint) {
        return balances[msg.sender] / 1 ether;
    }
}

```

<!-- How to Perform in Practical

Step 1. Open Remix IDE

Step 2. Create File

SimpleBank.sol

Step 3. Compile

Step 4. Deploy

Step 5. Test Functions

1. Deposit Money

Enter value in VALUE field (example: 1 Ether)

Click deposit

2. Check Balance

Click checkBalance

It will show your deposited amount.

3. Withdraw Money

Enter amount in Wei

Example:

1 Ether=10¹⁸ Wei

For 1 Ether withdrawal:

1000000000000000000

4. Click withdraw -->

Q12 Supply Chain Management

Problem Statement:

Create a smart contract to track products from manufacturer to consumer. Each step in the supply chain should be recorded immutably with timestamps and participant addresses.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract SupplyChain {

    struct Product {
        uint id;
        string name;
        address currentOwner;
        bool exists;
    }

    struct Tracking {
        address participant;
        string location;
        uint timestamp;
    }

    uint public productCount;

    mapping(uint => Product) public products;
    mapping(uint => Tracking[]) public productHistory;

    // Add new product
    function addProduct(string memory _name) public {

        productCount++;

        products[productCount] = Product(
            productCount,
            _name,
            msg.sender,
            true
        );

        productHistory[productCount].push(
            Tracking(
                msg.sender,
                "Manufactured",
                block.timestamp
            )
        );
    }
}
```

```

    )
  );
}

// Transfer product ownership
function transferProduct(
  uint _id,
  address _newOwner,
  string memory _location
) public {

  require(products[_id].exists, "Product does not exist");

  require(
    products[_id].currentOwner == msg.sender,
    "Only current owner can transfer"
  );

  products[_id].currentOwner = _newOwner;

  productHistory[_id].push(
    Tracking(
      _newOwner,
      _location,
      block.timestamp
    )
  );
}

// Get tracking count
function getTrackingCount(uint _id)
  public
  view
  returns(uint)
{
  return productHistory[_id].length;
}

// Get tracking details
function getTracking(
  uint _id,
  uint index
)
  public
  view
  returns(
    address,
    string memory,
    uint
  )

```

```

    {
        Tracking memory t = productHistory[_id][index];

        return (
            t.participant,
            t.location,
            t.timestamp
        );
    }
}

```

<!-- How to Perform in Practical

Step 1. Open Remix IDE

Step 2. Create File

SupplyChain.sol

Step 3. Compile

Step 4. Deploy

Step 5. Test Functions

1. Add Product

Input

Laptop

Action

Call:

addProduct("Laptop")

Expected Result

Product created

Owner becomes deployer address

First tracking entry stored:

"Manufactured"

timestamp

manufacturer address

2. Check Product Details

Call:

products(1)

Output

Product ID

Product name

Current owner address

Exists = true

3. Transfer Product to Distributor

Input

Product ID = 1

New owner = another MetaMask account

Location = "Distributor"

Action

Call:

transferProduct(1, address, "Distributor")

Expected Result

Ownership transferred

Blockchain records:

new participant address

location

timestamp

4. Transfer to Retailer

Switch MetaMask to distributor account.

Call:

transferProduct(1, retailerAddress, "Retailer")

Expected Result

Retailer becomes owner

New immutable tracking entry added

5. View Tracking History

Call:

getTrackingCount(1)

Suppose output:

3

Now check each record:

getTracking(1,0)

getTracking(1,1)

getTracking(1,2)

Sample Output

| Step | Participant Address | Location | Timestamp |
|------|---------------------|--------------|-----------|
| 0 | Manufacturer | Manufactured | Stored |
| 1 | Distributor | Distributor | Stored |
| 2 | Retailer | Retailer | Stored |

Q13 NFT Minting Contract

Problem Statement:

Develop a smart contract that allows users to mint and transfer non-fungible tokens (NFTs)

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.18;

/*
NFT Minting Contract
Features:
1. Mint NFTs
2. Transfer NFTs
3. Store metadata URI for each NFT
4. Track ownership
*/

contract NFTMinting {

    // NFT structure
    struct NFT {
        uint256 tokenId;
        string metadataURI;
        address owner;
    }

    // Token counter
    uint256 public nextTokenId = 1;

    // tokenId => NFT
    mapping(uint256 => NFT) public nfts;

    // owner => list of token IDs
    mapping(address => uint256[]) public ownerTokens;

    // Mint NFT
    function mintNFT(string memory _metadataURI) public {

        uint256 tokenId = nextTokenId;

        nfts[tokenId] = NFT({
            tokenId: tokenId,
            metadataURI: _metadataURI,
            owner: msg.sender
        });

        ownerTokens[msg.sender].push(tokenId);

        nextTokenId++;
    }

    // Transfer NFT
    function transferNFT(address _to, uint256 _tokenId) public {

        require(nfts[_tokenId].owner == msg.sender,

```

```

        "You are not the owner");

    address previousOwner = msg.sender;

    // Change ownership
    nfts[_tokenId].owner = _to;

    // Add token to new owner
    ownerTokens[_to].push(_tokenId);

    // Remove token from previous owner
    removeToken(previousOwner, _tokenId);
}

// Internal function to remove token
function removeToken(address _owner, uint256 _tokenId) internal {

    uint256[] storage tokens = ownerTokens[_owner];

    for(uint i = 0; i < tokens.length; i++) {

        if(tokens[i] == _tokenId) {

            tokens[i] = tokens[tokens.length - 1];
            tokens.pop();
            break;
        }
    }
}

// View NFT details
function getNFT(uint256 _tokenId)
    public
    view
    returns(uint256, string memory, address)
{

    NFT memory nft = nfts[_tokenId];

    return (
        nft.tokenId,
        nft.metadataURI,
        nft.owner
    );
}

// Get all NFTs owned by user
function getMyNFTs()
    public
    view

```



```

        returns(uint256[] memory)
    {
        return ownerTokens[msg.sender];
    }
}

```

<!-- How to Perform in Practical

Step 1. Open Remix IDE

Step 2. Create File

NFTMinting.sol

Step 3. Compile

Step 4. Deploy

Step 5. Test Functions

1: Mint NFT

Under deployed contract:

Function:

mintNFT

Input example:

<https://mynftsite.com/metadata/nft1.json>

Click transact.

Result:

NFT created

Owner = your account

Token ID = 1

2: Check NFT Details

Use:

getNFT(1)

Output:

tokenId = 1

metadata URI

owner address

3: Check Your NFTs

Call:

getMyNFTs()

Output:

[1]

Meaning:

You own NFT #1.

4: Transfer NFT

Change Account

In Remix/MetaMask keep another account ready.

Call:

transferNFT

Inputs:

_to = second account address

_tokenId = 1

Click transact.

NFT ownership transfers.

5: Verify Transfer

Call:

getNFT(1)

Owner address will now be the second account.

Also switch to second account and call:

getMyNFTs()

Output: [1]

Expected Viva Questions

1. What is NFT?

An NFT (Non-Fungible Token) is a unique digital asset stored on blockchain representing ownership.

2. What is metadata URI?

It stores information about NFT such as:

image

name

description

Why is tokenId unique?

Each NFT must have a unique identifier for ownership tracking.

3. What is msg.sender?

Address of the user calling the smart contract function.

4. Difference between NFT and Cryptocurrency?

NFT

Unique

Cannot replace each other

Represents ownership

representing digital assets.

Cryptocurrency

Fungible

Interchangeable

Represents currency

Include metadata for each NFT and ensure ownership

tracking.

Q14 Subscription Service

Problem Statement:

Implement a smart contract that manages subscription-based access to a service. Users should pay to subscribe for a period of time, and access should automatically expire afterward.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.18;
```

```
contract SubscriptionService {

    address public owner;
    uint public subscriptionFee = 1 ether;
    uint public subscriptionDuration = 1 minutes;

    struct Subscriber {
        uint expiryTime;
        bool subscribed;
    }

    mapping(address => Subscriber) public subscribers;

    constructor() {
        owner = msg.sender;
    }

    // Subscribe to the service
    function subscribe() public payable {
        require(msg.value == subscriptionFee, "Incorrect subscription fee");

        subscribers[msg.sender] = Subscriber({
            expiryTime: block.timestamp + subscriptionDuration,
            subscribed: true
        });
    }

    // Check if subscription is active
    function isSubscriptionActive(address user) public view returns (bool) {
        if (
            subscribers[user].subscribed &&
            block.timestamp <= subscribers[user].expiryTime
        ) {
            return true;
        } else {
            return false;
        }
    }
}
```

```

// Get remaining subscription time
function getRemainingTime(address user) public view returns (uint) {
    if (block.timestamp >= subscribers[user].expiryTime) {
        return 0;
    }

    return subscribers[user].expiryTime - block.timestamp;
}

// Owner withdraws collected funds
function withdrawFunds() public {
    require(msg.sender == owner, "Only owner can withdraw");

    payable(owner).transfer(address(this).balance);
}
}

```

<!-- How to Perform in Practical

Step 1. Open Remix IDE

Step 2. Create File

SubscriptionService.sol

Step 3. Compile

Step 4. Deploy

Step 5. Test Functions

A. Subscribe to Service

Under deployed contract:

Function:

subscribe

Value:

Enter:

1 ether

Then click transact.

This activates subscription for 30 days.

B. Check Subscription Status

Function:

isSubscriptionActive

Input your wallet address.

Output:

true

Means subscription is active.

C. Check Remaining Time

Function:

`getRemainingTime`

Input your address.

Output:

Time remaining in seconds.

D. Verify Automatic Expiry

Then call:

`isSubscriptionActive`

Output becomes:

`false`

This shows automatic expiration after time period.

E. Withdraw Funds (Owner Only)

Call:

`withdrawFunds`

Only deployer account can withdraw collected ether.

VIVA QUESTIONS

1. What is a subscription smart contract?

A blockchain contract that grants temporary access after payment.

2. What is `block.timestamp`?

Current blockchain timestamp used for expiry calculation.

3. Why use mapping?

To store subscription details for each user address.

4. How is expiration handled?

By comparing current time with stored expiry time.

5. Who can withdraw funds?

Only the contract owner.

Q15 Escrow Payment System

Problem Statement:

Design a smart contract that acts as an escrow between a buyer and a seller. Funds should

be released only when both parties agree or after a certain time if no disputes are raised.

// SPDX-License-Identifier: MIT

pragma solidity ^0.8.0;

contract EscrowPayment {

 address public buyer;
 address payable public seller;
 address public arbiter;

 uint public amount;
 uint public releaseTime;

 bool public buyerApproved;
 bool public sellerApproved;
 bool public fundsReleased;
 bool public disputeRaised;

 constructor(address payable _seller, uint _durationInMinutes) payable {
 buyer = msg.sender;
 seller = _seller;
 arbiter = msg.sender;

 amount = msg.value;

 // Auto release time
 releaseTime = block.timestamp + (_durationInMinutes * 1 minutes);
 }

 // Buyer approves payment release
 function approveByBuyer() public {
 require(msg.sender == buyer, "Only buyer can approve");
 buyerApproved = true;

 releaseFunds();
 }

 // Seller approves completion
 function approveBySeller() public {
 require(msg.sender == seller, "Only seller can approve");
 sellerApproved = true;

 releaseFunds();
 }

```

// Raise dispute before deadline
function raiseDispute() public {
    require(
        msg.sender == buyer || msg.sender == seller,
        "Not authorized"
    );

    require(block.timestamp < releaseTime, "Time expired");

    disputeRaised = true;
}

// Internal function to release funds
function releaseFunds() internal {

    require(!fundsReleased, "Funds already released");
    require(!disputeRaised, "Dispute exists");

    // Release if both approved
    if (buyerApproved && sellerApproved) {
        fundsReleased = true;
        seller.transfer(address(this).balance);
    }

    // Auto release after deadline
    else if (block.timestamp >= releaseTime) {
        fundsReleased = true;
        seller.transfer(address(this).balance);
    }
}

// Anyone can trigger auto release after deadline
function autoRelease() public {
    releaseFunds();
}

// View contract balance
function getBalance() public view returns(uint) {
    return address(this).balance;
}
}

```

<!-- How to Perform in Practical

Step 1. Open Remix IDE

Step 2. Create File
EscrowPayment.sol

Step 3. Compile

Step 4. Deploy

Step 5. Test Functions

1. Check Initial Details

After deployment verify:

buyer address
seller address
amount
releaseTime
balance

Click:

buyer()
seller()
getBalance()

2. Buyer Approval

Using buyer account:

Click:

approveByBuyer()

Now:

buyerApproved = true

But funds will NOT release yet because seller approval is pending.

3. Seller Approval

Switch account in Remix to seller account.

Click:

approveBySeller()

Now:

both approvals become true
Ether transfers to seller automatically

Check:

fundsReleased = true
getBalance() = 0

4. Test Dispute System

Deploy again.

Before time expires:

Click:

raiseDispute()

Now:
disputeRaised = true

If anyone tries:
approveByBuyer()
approveBySeller()
autoRelease()

transaction will fail because dispute exists.

5. Test Automatic Release After Time

Deploy again with:
1 minute
Wait 1–2 minutes.

Then click:
autoRelease()

Funds automatically transfer to seller if no dispute exists.

VIVA QUESTIONS:

Q1. What is escrow?

Escrow is a temporary holding mechanism where funds are stored securely until conditions are satisfied.

Q2. Why use smart contracts for escrow?

Because blockchain provides:

- transparency
- immutability
- automatic execution
- trustless transactions

Q3. What is block.timestamp?

It gives the current blockchain time in seconds.

Q4. Why use payable?

To allow the contract and seller address to receive Ether.

Q5. What happens if dispute is raised?

Funds remain locked until manual resolution (or additional logic is added).

Q16 Decentralized Lottery System

Problem Statement:

Create a lottery smart contract where users can buy tickets, and a random winner is picked

at the end of the round. Ensure fairness and prevent manipulation.

```
// SPDX-License-Identifier: MIT
```

```
pragma solidity ^0.8.0;
```

```
contract Lottery {
```

```
    address public manager;
```

```
    address[] public players;
```

```
    constructor() {
```

```
        manager = msg.sender;
```

```
    }
```

```
    // Buy lottery ticket
```

```
    function enter() public payable {
```

```
        require(msg.value == 1 ether, "Ticket price is 1 ETH");
```

```
        players.push(msg.sender);
```

```
    }
```

```
    // Generate random number
```

```
    function random() private view returns(uint) {
```

```
        return uint(
```

```
            keccak256(
```

```
                abi.encodePacked(
```

```
                    block.timestamp,
```

```
                    block.prevrandao,
```

```
                    players.length
```

```
                )
```

```
            )
```

```
        );
```

```
    }
```

```
    // Pick winner
```

```
    function pickWinner() public {
```

```
        require(msg.sender == manager, "Only manager can pick winner");
```

```
        require(players.length > 0, "No players entered");
```

```
        uint index = random() % players.length;
```

```
        payable(players[index]).transfer(address(this).balance);
```

```
        players = new address[](0);
```

```
    }
```

```
// View players
function getPlayers() public view returns(address[] memory) {
    return players;
}
}
```

<!-- How to Perform in Remix + MetaMask

1. Open Remix

2. Create File

Lottery.sol

Paste the code.

3. Compile

4. Connect MetaMask

5. Deploy Contract

6. Buy Lottery Tickets Using Different Accounts

Account 1

Select Account 1 in MetaMask

In VALUE field enter:

1

and choose ether

Click:

enter

Account 2

Repeat same steps.

Account 3

Repeat same steps.

Each account becomes a lottery participant.

7. Verify Players

Click:

getPlayers

You will see all participant wallet addresses.

8. Pick Winner

Switch back to manager account (deployer account).

Click:

pickWinner

The contract:
randomly selects winner
transfers all ETH to winner
resets player list

9. Verify Winner

Check:
MetaMask balance increase
Contract balance becomes 0
getPlayers() returns empty array
How Fairness is Ensured

VIVA QUESTIONS:

1. Why payable is used?

To allow contract to receive ETH.

2. Why require is used?

To validate conditions like:
correct ticket price
only manager can pick winner
Why randomness is not fully secure here?

3. Blockchain values like timestamp can sometimes be influenced slightly.

This is a basic educational lottery contract.

4. How to improve randomness?

Using:
Chainlink VRF
commit-reveal schemes
oracle-based randomness -->

Q17 Certificate Issuance and Verification

Problem Statement:

Build a smart contract that issues digital certificates (e.g., for a course or event) and allows

anyone to verify their authenticity using the certificate ID or address.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
```

```
contract CertificateVerification {

    address public admin;

    constructor() {
        admin = msg.sender;
    }
}
```

```

struct Certificate {
    string studentName;
    string courseName;
    uint issueDate;
    address issuedTo;
    bool valid;
}

mapping(uint => Certificate) public certificates;

// Issue Certificate
function issueCertificate(
    uint _certId,
    string memory _studentName,
    string memory _courseName,
    address _issuedTo
) public {

    require(msg.sender == admin, "Only admin can issue certificates");
    require(!certificates[_certId].valid, "Certificate ID already exists");

    certificates[_certId] = Certificate(
        _studentName,
        _courseName,
        block.timestamp,
        _issuedTo,
        true
    );
}

// Verify Certificate
function verifyCertificate(uint _certId)
    public
    view
    returns (
        string memory,
        string memory,
        uint,
        address,
        bool
    )
{
    Certificate memory cert = certificates[_certId];

    return (
        cert.studentName,
        cert.courseName,
        cert.issueDate,
        cert.issuedTo,

```

```

        cert.valid
    );
}
}

```

<!-- How to Perform in Practical

1. Open Remix

2. Create File

CertificateVerification.sol

Paste the code.

3. Compile

4. Connect MetaMask

5. Deploy Contract

Testing Steps for Practical

A. Issue Certificate

Expand deployed contract.

In issueCertificate enter:

```

    _certId → 1
    _studentName → "Rahul Sharma"
    _courseName → "Blockchain Course"
    _issuedTo → student wallet address

```

Click issueCertificate

Approve MetaMask transaction.

B. Verify Certificate

In verifyCertificate enter:

1

Click button.

You will get:

Student name

Course name

Issue date

Wallet address

true meaning valid certificate

VIVA QUESTIONS:

1. Why blockchain for certificates?

Because records cannot be modified easily and verification is transparent.

2. What is mapping?

It stores certificate data using certificate ID as key.

3. Why use block.timestamp?
To store certificate issue time.

4. What does require do?
It validates conditions and stops invalid transactions.

5. Who is admin?
The MetaMask account that deploys the contract.